



INFORMATIQUE

Sequence 6 : Types structurés

TD 6.1

1h30 -
v2025-09-
24 ⓘ

IUT d'Annecy, 9 rue de l'Arc en Ciel, 74940 Annecy

INTRODUCTION AUX TYPES STRUCTURÉS

Exercice 1 : Le problème sans structure

On souhaite gérer les informations d'un étudiant :

- son prénom,
- son âge,
- sa moyenne.

Question 1 Proposer une liste de variables pour stocker ces informations.

Par exemple :

```
string prenom;  
int age;  
float moyenne;
```

On peut discuter de la taille du tableau de caractères (ici 30).

Question 2 Écrire un code qui définit puis affiche les informations d'une étudiante nommée « Alice », 20 ans, moyenne 14.2.

```
string prenom = "Alice";  
int age = 20;  
float moyenne = 14.2f;  
5 printf("%s, %d ans, moyenne = %.1f\\n", prenom, age, moyenne);
```

Question 3 Comment stocker les informations de 3 étudiants différents ?

Sans structure, on peut multiplier les variables :

```
string prenom1, prenom2, prenom3;  
int age1, age2, age3;  
float moyenne1, moyenne2, moyenne3;
```

On peut aussi utiliser 3 tableaux parallèles :

```
string prenom[3];  
int ages[3];  
float moyennes[3];
```

Mais on voit déjà que cela devient plus difficile à suivre.

Question 4 Pourquoi cette approche devient-elle source d'erreurs ?

*Parce qu'il faut constamment **se souvenir** que les éléments exttprenom1, age1, moyenne1 vont ensemble, etc. On risque facilement de mélanger les indices ou d'oublier un champ. On ne manipule jamais « un*

étudiant » mais « plusieurs variables séparées », ce qui augmente la probabilité d'erreurs et rend le code difficile à maintenir.



Les types structurés en C

En C, on peut regrouper plusieurs variables dans une seule entité appelée **structure**, définie avec le mot-clé **struct**.

Dans notre cas, nous allons créer un type structuré qui contiendra différentes informations concernant une même entité.

La définition d'un type structuré se fait généralement avec le mot-clé **typedef** pour simplifier son utilisation ultérieure. La syntaxe générale est la suivante :

```
typedef struct {  
    type1 champ1;  
    type2 champ2;  
    ...  
5    typeN champN;  
} NomDeLaStructure;
```

Prenons l'exemple d'un contact dans un carnet d'adresses. On peut définir une structure **Contact** qui regroupe plusieurs champs :

```
typedef struct {  
    string nom;  
    string prenom;  
    string telephone;  
    string email;  
5    int anneeNaissance;  
} T_Contact;
```

Exercice 2 : Créer une structure cohérente

On regroupe les données dans une seule entité : un **type structuré**.

Question 1 À l'aide de l'exemple précédent, écris la définition d'une structure **Etudiant** contenant les champs correspondants aux informations d'un étudiant (prénom, âge, moyenne).

*Par exemple, avec un **typedef** :*

```
typedef struct {  
    string prenom;  
    int age;  
    float moyenne;  
5 } T_Etudiant;
```

Question 2 Écrire les déclarations suivantes :

- une variable **etudiant_1** représentant un étudiant,
- un tableau **classe** de 30 étudiants.

```
T_Etudiant etudiant_1;  
T_Etudiant classe[30];
```

Question 3 Écrire les lignes de code permettant d'initialiser les informations de l'étudiant **etudiant_1** avec les valeurs suivantes : prénom « Bob », âge 22, moyenne 15.5.

Deux possibilités :

Initialisation directe :

```
| T_Etudiant etudiant_1 = {"Bob", 22, 15.5f};
```

Affectation champ par champ :

```
| etudiant_1.prenom = "Bob";
| etudiant_1.age = 22;
| etudiant_1.moyenne = 15.5f;
```

Question 4 Écrire les lignes de code permettant de demander à l'utilisateur de saisir les informations des 30 étudiants du tableau `classe`.

```
5 | int i;
   | for (i = 0; i < 30; i++) {
   |     printf("Etudiant %d\\n", i+1);
   |     classe[i].prenom = get_string("Prenom : ");
   |
   |     classe[i].age = get_int("Age : ");
   |
   |     classe[i].moyenne = get_float("Moyenne : ");
10 | }
```

Question 5 Écrire les lignes de code permettant d'afficher le prénom et la moyenne du meilleur étudiant de la classe.

```
5 | int i, indice_meilleur = 0;
   | for (i = 1; i < 30; i++) {
   |     if (classe[i].moyenne > classe[indice_meilleur].moyenne) {
   |         indice_meilleur = i;
   |     }
   | }
10 | printf("Meilleur etudiant : %s, moyenne = %.2f\\n",
   |     classe[indice_meilleur].prenom,
   |     classe[indice_meilleur].moyenne);
```

Exercice 3 : Représenter une heure

On veut manipuler une heure sous la forme HH:MM:SS.

Question 1 Proposer un type structuré `T_Heure` permettant de stocker les informations d'un horaire.

```
5 | typedef struct {
   |     int h;
   |     int m;
   |     int s;
   | } T_Heure;
```

Question 2 Écrire une fonction `void afficher_heure(T_Heure t)` qui affiche HH:MM:SS.

```
| void afficher_heure(T_Heure t) {
|     printf("%02d:%02d:%02d\\n", t.h, t.m, t.s);
| }
```

Question 3 Écrire `int en_secondes(T_Heure t)` qui renvoie le nombre total de secondes.

```
int en_secondes(T_Heure t) {  
    return t.h * 3600 + t.m * 60 + t.s;  
}
```

Question 4 Créer deux horaires dans le `main` et afficher la différence en secondes.

```
int main(void) {  
    T_Heure h1 = {8, 30, 0};  
    T_Heure h2 = {10, 15, 30};  
  
5    int s1 = en_secondes(h1);  
    int s2 = en_secondes(h2);  
  
    printf("Différence = %d secondes\n", s2 - s1);  
    return 0;  
10 }
```

Question 5 Ecrire la fonction inverse qui prend en paramètre un entier représentant un nombre de secondes et qui renvoie une structure `T_Heure` correspondante.

```
T_Heure inverse(int total_secondes) {  
    T_Heure t;  
    t.h = total_secondes / 3600;  
    total_secondes %= 3600;  
5    t.m = total_secondes / 60;  
    t.s = total_secondes % 60;  
    return t;  
}
```

Exercice 4 : Gérer un produit en stock

On veut garder les informations suivantes pour chaque produit :

- son nom (chaîne de caractères),
- son prix HT (nombre à virgule),
- son taux de TVA (nombre à virgule).

Question 1 Proposer un type structuré `T_Produit` pour stocker ces informations.

```
typedef struct {  
    string nom;  
    float prixHT;  
    float tva; // en pourcentage, par ex. 20.0  
5 } T_Produit;
```

Question 2 Écrire une fonction qui renvoie le prix TTC d'un produit (la fonction ne prendra qu'un seul argument).

```
float prixTTC(T_Produit p) {  
    return p.prixHT * (1.0f + p.tva / 100.0f);  
}
```

On pourra ensuite utiliser cette fonction dans un `main` pour comparer plusieurs produits.

Exercice 5 : Gestion d'un vecteur 2D

On manipule des vecteurs dans un plan :

```
typedef struct{  
    float x;  
    float y;  
} T_Vecteur;
```

Question 1 Écrire une fonction qui renvoie la norme $\sqrt{x^2 + y^2}$ d'un vecteur donné en paramètre.

```
#include <math.h>  
  
float norme(T_Vecteur v) {  
    return sqrtf(v.x * v.x + v.y * v.y);  
5 }
```

Question 2 Écrire une fonction permettant de sommer deux vecteurs.

```
T_Vecteur addition(T_Vecteur a, T_Vecteur b) {  
    T_Vecteur res;  
    res.x = a.x + b.x;  
    res.y = a.y + b.y;  
5    return res;  
}
```

Question 3 Écrire une fonction permettant de donner l'opposé d'un vecteur.

```
T_Vecteur oppose(T_Vecteur v) {  
    T_Vecteur res;  
    res.x = -v.x;  
    res.y = -v.y;  
5    return res;  
}
```

Question 4 Dans le main, déclarer trois vecteurs, afficher leurs normes, puis afficher la norme de $a - b$.

```
int main(void) {  
    T_Vecteur a = {3.0f, 4.0f};  
    T_Vecteur b = {1.0f, 2.0f};  
    T_Vecteur c = {-2.0f, 0.5f};  
5  
    printf("||a|| = %f\n", norme(a));  
    printf("||b|| = %f\n", norme(b));  
    printf("||c|| = %f\n", norme(c));  
  
10    T_Vecteur diff;  
    diff.x = a.x - b.x;  
    diff.y = a.y - b.y;  
  
    printf("||a-b|| = %f\n", norme(diff));  
15    return 0;  
}
```

Exercice 6 : Extension : struct imbriquées

Il est également possible de créer des structures imbriquées, c'est-à-dire d'utiliser une structure comme champ d'une autre structure. Par exemple, on peut définir une structure `Adresse` et l'utiliser dans une structure `Etudiant` :

```
typedef struct {  
    string rue;  
    int codePostal;  
    string ville;  
5 } T_Adresse;  
  
typedef struct {  
    string prenom;  
    int age;  
10 float moyenne;  
    T_Adresse adresse;  
} T_EtudiantAdresse;
```

Question 1 Comment accéder à la ville d'un étudiant ?

Si `e` est une variable de type `T_EtudiantAdresse`, on accède à la ville par :

```
| e.adresse.ville
```

Question 2 Quel est l'avantage de créer une structure imbriquée plutôt qu'ajouter 3 champs à `Etudiant` ?

On regroupe tous les champs liés à l'adresse dans une seule structure `T_Adresse`. Cela permet :

- de réutiliser `T_Adresse` pour d'autres structures (*Professeur, Entreprise, etc.*);
- de rendre le code plus lisible (une entité logique = une structure);
- de faciliter les modifications (ajouter un champ à l'adresse sans toucher toutes les structures qui l'utilisent).

Question 3 Proposer une fonction `int habite_ville(T_EtudiantAdresse e, const char* ville)` qui renvoie 1 si l'étudiant habite dans la ville donnée, 0 sinon.

```
| int habite_ville(T_EtudiantAdresse e, const char* ville) {  
|     return (strcmp(e.adresse.ville, ville) == 0);  
| }
```

On peut ensuite parcourir un tableau d'étudiants et afficher ceux qui habitent dans une ville donnée.